"Se você realmente quer aprender, você deve construir a máquina e se acostumar com suas excentricidades pela prática (Wilbur Wright)."

Criando APIs

Paulo Lisboa de Almeida

GenAI

UFPR

API

Application Programming Interface (API).

Expor funções para que programas (seus e de terceiros) usem.

- Interface.
- Conexões locais ou remotas.
  - Depende do problema.

Ambiente

Crie um novo ambiente conda para o projeto.

```
conda create --name nome_ambiente
```

Ative o ambiente

```
conda activate nome_ambiente
```

Atenção: abriremos vários terminais. Garanta que você está no ambiente Conda correto nos terminais.

Consumindo

Get

Podemos solicitar dados de servidores através de uma solicitação do tipo GET.

GET: retorne um dado, sem modificar o estado interno das informações.

Solicitação via URL, passando os parâmetros diretamente (query parameters).

Exemplo: teste em seu navegador.

https://geocoding-api.open-meteo.com/v1/search?name=curitiba&count=1

Get

https://geocoding-api.open-meteo.com/v1/search?name=curitiba&count=1

Parâmetros esperados pela função. Estamos buscando por Curitiba, e o máximo de respostas esperadas é 1.

Método (função) de busca (*search*) por uma cidade. Vai retornar latitute e longitude (dentre outros).

Chamando a API do open-meteo -> https://open-meteo.com

API via Python

Podemos fazer requisições de APIs via Python.

Para isso, instale requests no seu ambiente Conda:

```
conda install requests
```

Consumidor

Vamos criar um script que orquestra as chamadas de uma API.

Chame de `consumidor.py`.

Consumidor.py

Endereço completo da requisição, **sem** os parâmetros.

```
import requests

cidade = "Curitiba"

geo_url = "https://geocoding-api.open-meteo.com/v1/search"

geo_params = {
    "name": cidade,
    "count": 1
}

geo_response = requests.get(geo_url, params=geo_params)
geo_data = geo_response.json()

lat = geo_data["results"][0]["latitude"]
lon = geo_data["results"][0]["longitude"]

print("Lat:", lat, "Lon:", lon)
```

Os parâmetros são enviados através de um dicionário.

Consumidor.py

```python
import requests

cidade = "Curitiba"

geo_url = "https://geocoding-api.open-meteo.com/v1/search"

geo_params = {
    "name": cidade,
    "count": 1
}

geo_response = requests.get(geo_url, params=geo_params)
geo_data = geo_response.json()

lat = geo_data["results"][0]["latitude"]
lon = geo_data["results"][0]["longitude"]

print("Lat:", lat, "Lon:", lon)
```

O primeiro comando faz a requisição. O segundo pega o json recebido, e o transforma em um dicionário Python.

Dica: faça print(geo_data) para entender a estrutura do Json. Isso vai esclarecer os comandos seguintes.

Consumidor.py

```python
import requests

cidade = "Curitiba"

geo_url = "https://geocoding-api.open-meteo.com/v1/search"

geo_params = {
    "name": cidade,
    "count": 1
}

geo_response = requests.get(geo_url, params=geo_params)
geo_data = geo_response.json()

lat = geo_data["results"][0]["latitude"]
lon = geo_data["results"][0]["longitude"]

print("Lat:", lat, "Lon:", lon)
```

Consumidor.py

Vamos usar os dados de latitude e longitude para criar chamar outra função da API.

Exibir a temperatura atual.

Adicione

```
weather_url = "https://api.open-meteo.com/v1/forecast"

weather_params = {
    "latitude": lat,
    "longitude": lon,
    "current_weather": True
}

weather_response = requests.get(weather_url, params=weather_params)
weather_data = weather_response.json()

print("Temperatura:", weather_data["current_weather"]["temperature"])
```

FastAPI

FastAPI

O FastAPI é uma forma simples de se criar APIs que podem ser servidas via requisições WEB.

https://fastapi.tiangolo.com

Comunicação comumente feita através de JSONs.

FastAPI

Ambiente

Instale o FastAPI

```
conda install fastapi
```

Olá mundo

Crie um arquivo servidor.py com o seguinte:

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/')
def saudacao():
    return {'saudacao': 'Olá Mundo!'}
```

Olá mundo

Crie um arquivo servidor.py com o seguinte:

```
from fastapi import FastAPI

app = FastAPI()

@app.get('/')
def saudacao():
    return {'saudacao': 'Olá Mundo!'}
```

Instância da classe FastAPI, que vai gerenciar as APIs.

Subindo o servidor localmente

Abra um terminal separado no mesmo diretório do arquivo **main.py**.

Não esqueça de carregar o ambiente Conda correto.

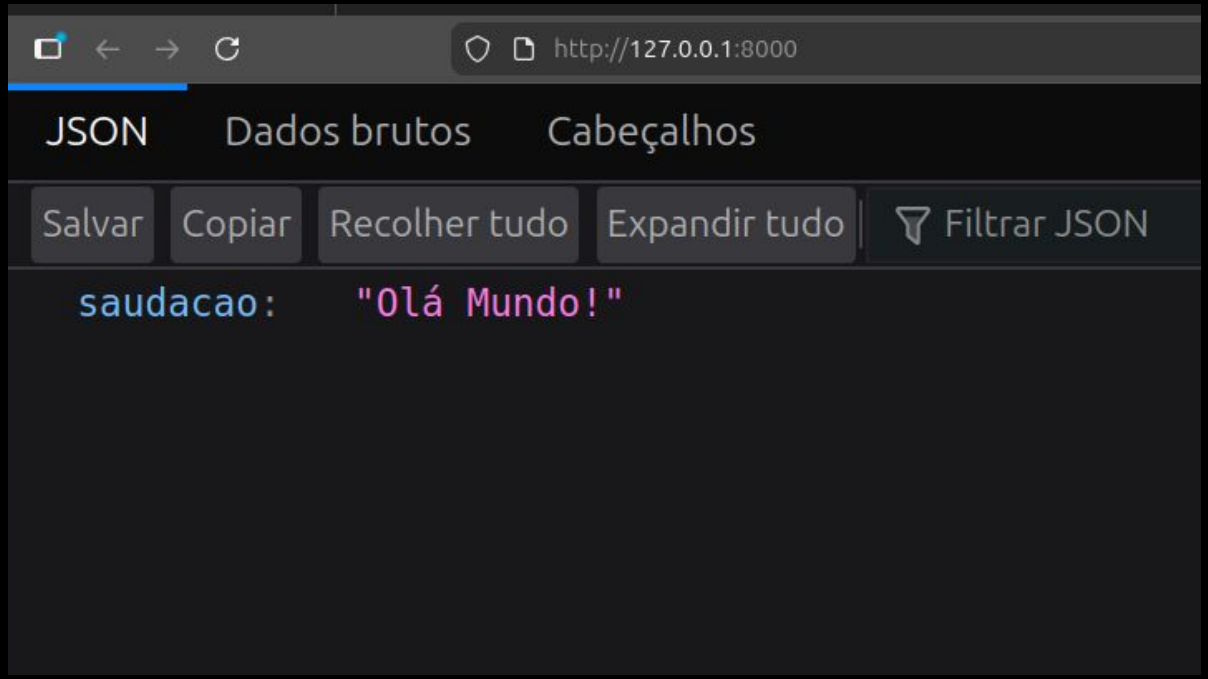
Execute `fastapi dev servidor.py`

Um servidor local será instanciado.

Acessando

Em um navegador qualquer, acesse o endereço:

http://127.0.0.1:8000

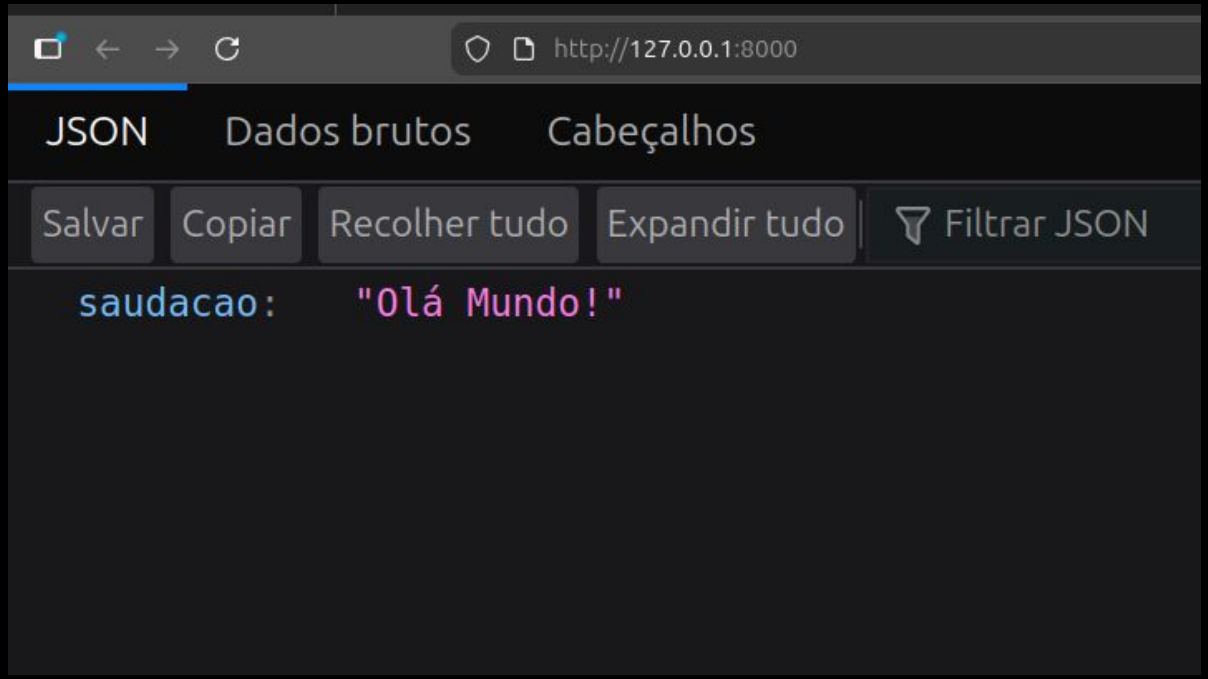


Acessando

Em um navegador qualquer, acesse o endereço:

http://127.0.0.1:8000

O resultado é que conseguimos acessar a função saudacao!



127.0.0.1

Pelo menos por enquanto, vamos sempre executar fastapi no modo dev.

Serviços acessíveis apenas na máquina local (127.0.0.1).

Muito cuidado ao prover serviços visíveis na internet.

Fácil abrir um *backdoor* (ou até um "*frontdoor*") sem perceber, e criar sistemas vulneráveis.

Endereços

```python
from fastapi import FastAPI
import datetime

app = FastAPI()

@app.get('/saudacao')
def saudacao():
    return {'saudacao': 'Olá Mundo!'}

@app.get('/hora_atual')
def hora_atual():
    return {'hora': str(datetime.datetime.now())}
```

Endereços

```python
from fastapi import FastAPI
import datetime

app = FastAPI()

@app.get('/saudacao')
def saudacao():
    return {'saudacao': 'Olá Mundo!'}

@app.get('/hora_atual')
def hora_atual():
    return {'hora': str(datetime.datetime.now())}
```

Acessível a partir de http://127.0.0.1:8000/saudacao

Parâmetros

Query Params

```python
@app.get('/verificar-cpf')
def verificar_cpf(cpf_teste):
    cpf_teste = int(cpf_teste)
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0

    ultimo = cpf_teste % 10
    cpf_teste //= 10
    penultimo = cpf_teste % 10
    cpf_teste //= 10

    somatorio_valida_ultimo = penultimo * 2
    for i in range(2, 12):
        modulo = cpf_teste % 10
        cpf_teste //= 10
        somatorio_valida_penultimo += modulo * i
        somatorio_valida_ultimo += modulo * (i + 1)

    modulo = somatorio_valida_penultimo % 11
    if modulo < 2:
        if penultimo != 0:
            return False  # CPF inválido
    else:
        if penultimo != 11 - modulo:
            return False  # CPF inválido

    modulo = somatorio_valida_ultimo % 11
    if modulo < 2:
        if ultimo != 0:
            return False  # CPF inválido
    else:
        if ultimo != 11 - modulo:
            return False  # CPF inválido
    return True  # CPF válido
```

Query Params

```python
@app.get('/verificar-cpf')
def verificar_cpf(cpf_teste):
    cpf_teste = int(cpf_teste)
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0

    ultimo = cpf_teste % 10
    cpf_teste //= 10
    penultimo = cpf_teste % 10
    cpf_teste //= 10

    somatorio_valida_ultimo = penultimo * 2
    for i in range(2, 12):
        modulo = cpf_teste % 10
        cpf_teste //= 10
        somatorio_valida_penultimo += modulo * i
        somatorio_valida_ultimo += modulo * (i + 1)

    modulo = somatorio_valida_penultimo % 11
    if modulo < 2:
        if penultimo != 0:
            return False  # CPF inválido
    else:
        if penultimo != 11 - modulo:
            return False  # CPF inválido

    modulo = somatorio_valida_ultimo % 11
    if modulo < 2:
        if ultimo != 0:
            return False  # CPF inválido
    else:
        if ultimo != 11 - modulo:
            return False  # CPF inválido
    return True  # CPF válido
```

O padrão é nomear recursos com nomes compostos separando com hífen (note que o nome da função Python é diferente). Leia mais sobre isso procurando sobre como são feitas arquiteturas REST.

Query Params

```python
@app.get('/verificar-cpf')
def verificar_cpf(cpf_teste):
    cpf_teste = int(cpf_teste)
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0

    ultimo = cpf_teste % 10
    cpf_teste //= 10
    penultimo = cpf_teste % 10
    cpf_teste //= 10

    somatorio_valida_ultimo = penultimo * 2
    for i in range(2, 12):
        modulo = cpf_teste % 10
        cpf_teste //= 10
        somatorio_valida_penultimo += modulo * i
        somatorio_valida_ultimo += modulo * (i + 1)

    modulo = somatorio_valida_penultimo % 11
    if modulo < 2:
        if penultimo != 0:
            return False  # CPF inválido
    else:
        if penultimo != 11 - modulo:
            return False  # CPF inválido

    modulo = somatorio_valida_ultimo % 11
    if modulo < 2:
        if ultimo != 0:
            return False  # CPF inválido
    else:
        if ultimo != 11 - modulo:
            return False  # CPF inválido
    return True  # CPF válido
```

Podemos declarar o parâmetro normalmente, mas precisamos converter para um int, já que a requisição enviará o `cpf_teste` como uma string.

Acessando

http://127.0.0.1:8000/verificar_cpf?cpf_teste=11111111111

Hint de Tipo

```python
@app.get('/verificar-cpf')
def verificar_cpf(cpf_teste: int):
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0
    #...
```

Podemos remover o cast e inserir um hint (dica) de tipo para o interpretador.

Cuidado: diferente de uma linguagem tipada, o hint só indica para o interpretador que nesse ponto a variável precisaria ser um inteiro. Nada impede que a variável mude de tipo daqui para frente.

Path param

```python
@app.get('/verificar-cpf/{cpf_teste}')
def verificar_cpf(cpf_teste: int):
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0
    #...
```

Path param

```python
@app.get('/verificar-cpf/{cpf_teste}')
def verificar_cpf(cpf_teste: int):
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0
    #...
```

http://127.0.0.1:8000/verificar-cpf/11111111111

O parâmetro faz parte do caminho do recurso.

docs

Docs

O FastAPI gera automaticamente páginas de documentação e testes.

Basta acessar 127.0.0.1:8000/docs

Faça testes nas funções que você criou até agora.

Anotando parâmetros

Podemos adicionar anotadores em parâmetros de Query e de Path.

As anotações servem para validação dos dados, e para documentação. Veja o exemplo:

```
@app.get('/verificar-cpf/{cpf_teste}')
def verificar_cpf(cpf_teste: Annotated[int, Path(title="CPF", description="O CPF a ser testado", ge=1, le=99999999999)]):
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0
    ...
```

Anotando parâmetros

GET /verificar-cpf/{cpf_teste} Verificar Cpf

Parameters

Cancel

Name	Description
cpf_teste * required integer (path)	O CPF a ser testado 11111111111 maximum: 99999999999 minimum: 1

Please correct the following validation errors and try again.

- For 'cpf_teste': Value must be less than or equal to 99999999999.

Execute | Clear

GET, POST, PUT, …

Get

Até agora fizemos requisições do tipo GET.

- Recuperar informações.
- **Nunca modificar** (não altera o estado).
- Idempotente: múltiplas requisições retornam a mesma resposta.

POST

POST

- Para inserir um novo conteúdo (alterar o estado do servidor), utilize um post.
- Exemplo: cadastrar um novo usuário ou produto em um banco de dados.
- POSTs **não** são idempotentes.

Fazendo um POST

Para simular um post (e um get), vamos salvar os usuários em uma lista simples.

```
from fastapi import FastAPI
import datetime

app = FastAPI()

dict_pessoas = {}

#...

@app.post("/adicionar-pessoa")
def adicionar_pessoa(cpf: int, nome: str):
     dict_pessoas[cpf] = (cpf, nome)


@app.get('/nome-pessoa/{cpf}')
def nome_pessoa(cpf: int):
   return dict_pessoas[cpf][1]
```

Fazendo um POST

Para simular um post (e um get), vamos salvar os usuários em uma lista simples.

```
from fastapi import FastAPI
import datetime

app = FastAPI()

dict_pessoas = {}

#...

@app.post("/adicionar-pessoa")
def adicionar_pessoa(cpf: int, nome: str):
     dict_pessoas[cpf] = (cpf, nome)


@app.get('/nome-pessoa/{cpf}')
def nome_pessoa(cpf: int):
   return dict_pessoas[cpf][1]
```

Estamos fazendo um POST.

Fazendo um POST

Para simular um post (e um get), vamos salvar os usuários em uma lista simples.

```
from fastapi import FastAPI
import datetime

app = FastAPI()

dict_pessoas = {}

#...

@app.post("/adicionar-pessoa")
def adicionar_pessoa(cpf: int, nome: str):
     dict_pessoas[cpf] = (cpf, nome)


@app.get('/nome-pessoa/{cpf}')
def nome_pessoa(cpf: int):
   return dict_pessoas[cpf][1]
```

Uma função de GET para testar.

Testando

Temos duas opções para testar:

1. Usando o docs
2. Criando uma aplicação cliente (ex.: em Python).

    Fica como exercício.

Testando

FastAPI 0.1.0 OAS 3.1

/openapi.json

default

GET /saudacao Saudacao

GET /hora_atual Hora Atual

GET /verificar-cpf/{cpf_teste} Verificar Cpf

POST /adicionar-pessoa Add Item

Parameters

Cancel

Name	Description
cpf * required integer (query)	11111111111
nome * required string (query)	José da Silva

Execute | Clear

Responses

Outros métodos

Existem vários outros métodos REST. Pesquise!

HTTP Method	Ação
POST	Criar (cadastrar) um item.
GET	Ler um dado (pode ser filtrado).
PUT	Atualizar um item.
PATCH	Atualização parcial de um item.
DELETE	Remover um item.

Cuidado!

É de responsabilidade do programador(a) programar os métodos REST corretamente do lado do servidor.

Por exemplo, você pode, mas **não deve** utilizar um GET para cadastrar items. Isso geraria.

- Brechas severas de segurança.
- Erros lógicos.
- Código difícil de manter.

HTTPException

HTTPException

Usando o conceito de exceções visto anteriormente, podemos lançar uma exceção com o FastAPI.

A exceção será tratada pelo FastAPI, que vai enviar a mensagem de erro para o cliente com o formato correto.

Você só precisa se preocupar com a mensagem, e com o código de erro.

Vamos usar 400 (Bad Request). Mas existem vários outros. Pesquise!

HTTPException

```python
from fastapi import FastAPI, Path, HTTPException
from typing import Annotated
import datetime

app = FastAPI()


dict_pessoas = {}



...

@app.post("/adicionar-pessoa")
def adicionar_pessoa(cpf: int, nome: str):
    if cpf in dict_pessoas:
        raise HTTPException(status_code=400, detail="CPF já cadastrado.")
    dict_pessoas[cpf] = (cpf, nome)

@app.get('/nome-pessoa/{cpf}')
def nome_pessoa(cpf: int):
    if cpf not in dict_pessoas:
        raise HTTPException(status_code=400, detail="Pessoa inexistente.")
    return dict_pessoas[cpf][1]
```

HTTPException

Faça testes no docs do FastAPI e veja os erros sendo enviados.

async

async

A palavra chave async na definição da função que será chamada pelo FastAPI indica que a chamada pode ser assíncrona.

Grosso modo, indica que o servidor pode começar a tratar outra requisição antes de terminar a anterior, e pode tratar as requisições na ordem que julgar melhor.

As chamadas não são síncronas.

+ Reduzimos o tempo de resposta, especialmente se as funções dependem de recursos que podem ser lentos (ex.: um GET que precisa abrir um arquivo).

- Precisamos tomar muito mais cuidado, já que podemos gerar condições de corrida.
- Algumas funções que chamamos dentro das nossas funções assíncronas podem não suportar chamadas assíncronas.

Teste

No servidor, crie a função:

```
@app.get('/teste_async')
async def teste(id: int):
    return "Teste " + str(id)
```

Cliente

Crie um cliente com o seguinte, e faça testes mudando adicionando/removendo async da função de testes sendo chamada.

```
import requests
import threading

URL = "http://127.0.0.1:8000/teste_async"

def fazer_requisicao(i):
    response = requests.get(URL, params={"id": i})
    print(response.text)

threads = []

for i in range(1, 101):
    t = threading.Thread(target=fazer_requisicao, args=(i,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()
```

Dica: lembre-se que para executar o cliente, basta executá-lo normalmente:
`python3 cliente.py`

Orquestrando com outras APIs

Orquestrando

Você pode criar uma API que chama e orquestra outras APIS para resolver determinado problema.

Por exemplo (e fica como exercício), criar uma função que recebe o nome de uma cidade, e chama as APIs do open-meteo corretamente para retornar os dados de clima da localização desejada.

Exercícios

1. Crie scripts Python para consumir os serviços que você criou.
2. Suba um serviço no FastAPI sem o "dev". Agora o serviço será visível a todos que estão na mesma rede.
    a. Crie um serviço simples, como os dados em aula, em seu computador. O seu colega deve criar um cliente que consome os serviços hospedados em sua máquina.
    b. Dica: o serviço será acessível a partir do IP interno do computador do seu colega.

Licença

Esta obra está licenciada com uma Licença Creative Commons Atribuição 4.0 Internacional.